Clase 13: Programación con memoria compartida

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Jueves 23 de Abril del 2026

Midiendo el rendimiento

Factores que afectan la escalabilidad

Patrones de paralelismo

Fuentes de degradación del rendimiento

Límites del paralelismo en memoria compartida

Midiendo el rendimiento

Factores que afectan la escalabilidad

Patrones de paralelismo

Fuentes de degradación del rendimiento

Límites del paralelismo en memoria compartida

Midiendo tiempo con omp_get_wtime

```
#include <omp.h>

double t0 = omp_get_wtime();

// ... código a medir ...

double t1 = omp_get_wtime();
printf("Tiempo: %.4f s\n", t1 - t0);
```

- Mide *wallclock time*: tiempo real transcurrido, no tiempo de CPU.
- Para medir *speedup*: ejecutar la versión serial y la paralela con los mismos datos.
- Repetir varias veces y usar el mínimo o promedio para reducir ruido.

Speedup y eficiencia

Dado un problema de tamaño fijo:

- T_1 = tiempo de ejecución con 1 *thread*.
- T_p = tiempo de ejecución con p *threads*.

Se definen:

$$S_p = \frac{T_1}{T_p} \qquad \text{(speedup)}$$

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p} \qquad \text{(eficiencia)}$$

- Speedup ideal: $S_p = p$, eficiencia ideal: $E_p = 1$.
- En la práctica: sincronización, parte serial y desbalance reducen ambas métricas.

Escalabilidad fuerte y débil

- Escalabilidad fuerte (*strong scaling*): mismo problema, más *threads*.
  - Objetivo: reducir el tiempo de ejecución.
  - La fracción del trabajo que no puede paralelizarse limita el speedup máximo.

- Escalabilidad débil (*weak scaling*): más *threads* con problema proporcionalmente más grande.
  - Objetivo: mantener el tiempo de ejecución constante al crecer el problema.
  - Más fácil de alcanzar cuando la carga por *thread* se mantiene constante.

- La escalabilidad débil no siempre aplica: hay problemas cuyo tamaño es fijo.

Midiendo el rendimiento

Factores que afectan la escalabilidad

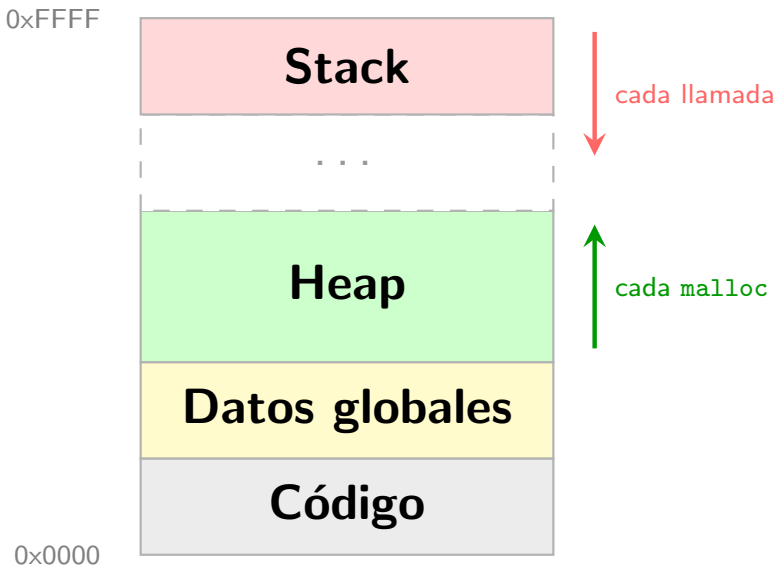
Patrones de paralelismo

Fuentes de degradación del rendimiento

Límites del paralelismo en memoria compartida

Espacio de memoria de un proceso

El SO divide la memoria de un proceso en regiones con propósitos distintos:



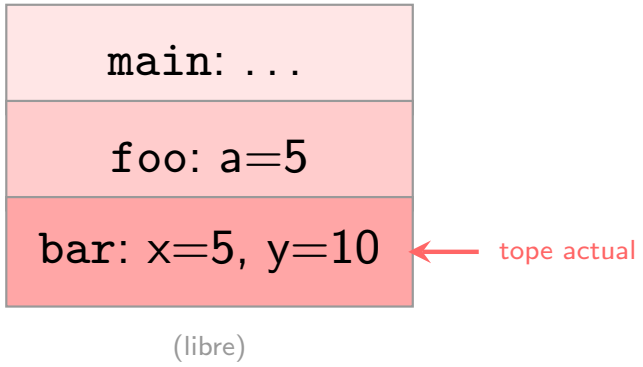
El stack se llena al entrar a una función (variables locales, parámetros) y se vacía automáticamente al retornar. El heap crece con `malloc`/`new` y se reduce con `free`/`delete`.

El stack: memoria automática por función

Cada llamada a función reserva un **frame** en el stack con sus variables locales. Al retornar, el frame desaparece.

```
void bar(int x) {
    int y = x * 2; // en stack
    // ...
}               // y desaparece

void foo() {
    int a = 5; // en stack
    bar(a);
}               // a desaparece
```



Cuando `bar` retorne, su frame se elimina y `x`, `y` dejan de existir. El stack es LIFO: siempre se libera el frame más reciente primero.

El heap: memoria manual y persistente

El heap permite reservar memoria cuya vida no está ligada a ninguna función.

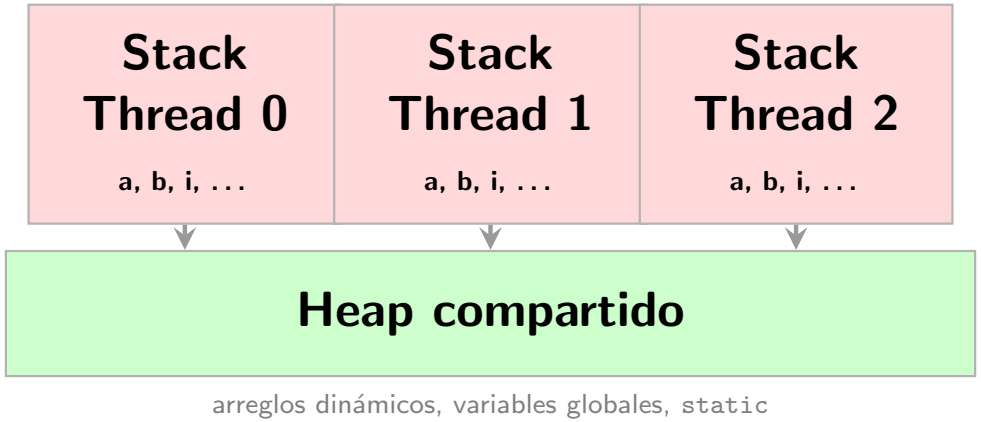
```
int* crear(int n) {
    // malloc: pide n*4 bytes
    int* v = malloc(n *
         sizeof(int));
    return v; // v persiste!
}

int main() {
    int* datos = crear(1000);
    // ... usar datos ...
    free(datos); // liberar
}
```

- `malloc` reserva un bloque en el heap y devuelve un puntero a él.
- El bloque sobrevive al retorno de `crear`: no está en el stack de esa función.
- Si se olvida el `free`: *memory leak*.
- El heap es compartido entre todos los *threads* del proceso.

Stack, heap y threads

Cada *thread* tiene su propio stack, pero todos comparten el mismo heap.



Una variable local `int i` en Thread 0 es independiente del `int i` en Thread 1: viven en stacks distintos. Un arreglo reservado con `malloc` es accesible desde cualquier thread.

Contención

Cuando varios *threads* intentan acceder al mismo recurso al mismo tiempo, compiten por él. A esa competencia se le llama **contención**.

- En una variable `shared` que múltiples *threads* escriben, la contención ocurre a nivel de hardware: solo un núcleo puede tener la línea de caché en estado Modified a la vez.
- En una sección `critical`, la contención ocurre a nivel de software: solo un *thread* entra al bloque a la vez. Los demás quedan bloqueados.

Visibilidad de variables y contención

En OpenMP toda variable tiene una **visibilidad**: `private` o `shared`.

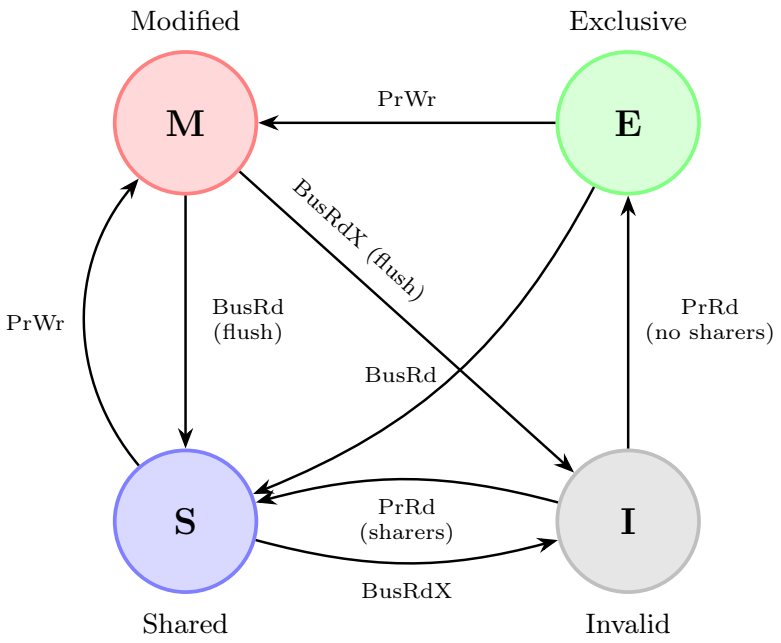
- `private`: cada *thread* tiene su propia copia en su *stack*. Sin dirección compartida, no hay contención de hardware.
- `shared`: todos los *threads* acceden a la misma dirección. El impacto depende del **patrón de acceso**: cuántos leen, cuántos escriben y con qué frecuencia.

Tipos de acceso a variables compartidas

Patrón de acceso	**Estado MESI**	**Escalabilidad**
`private` (sin compartir)	no aplica	perfecta
`shared`, solo lectura	Shared (S)	buena
`shared`, escritura desde 1 *thread*	Modified (M)	aceptable
`shared`, escritura desde varios	invalida en loop	mala
false sharing	invalida la línea entera	mala

La regla práctica es minimizar las variables `shared` que múltiples *threads* escriben. Cuando sea necesario combinar resultados, usar `reduction` en lugar de variables compartidas directas.

Modelo MESI



- **Modified** : línea modificada localmente, no escrita a memoria.
- **Exclusive** : única copia, pero *limpia* (igual a memoria).
- **Shared** : presente en múltiples cachés, no modificada.
- **Invalid** : línea no presente o invalidada porque otro core modificó el dato.

Ejemplo: cómo la visibilidad cambia el rendimiento

```
// Lento: suma sobre variable shared escrita por todos
double total = 0.0;
#pragma omp parallel for
for (int i = 0; i < N; i++)
    total += a[i]; // race condition + invalidaciones
```

```
// Rápido: acumulación private, reducción al final
double total = 0.0;
#pragma omp parallel for reduction(+:total)
for (int i = 0; i < N; i++)
    total += a[i]; // cada thread acumula en copia private
```

- Con `reduction`, cada *thread* acumula en una variable `private`. No hay invalidaciones durante el loop.
- La única sincronización ocurre al final, una sola vez por *thread*.

Midiendo el rendimiento

Factores que afectan la escalabilidad

Patrones de paralelismo

Fuentes de degradación del rendimiento

Límites del paralelismo en memoria compartida

¿Por qué pensar en patrones?

Un **patrón de paralelismo** es una estructura de cómputo recurrente con una forma canónica de paralelizarse.

- Identificar el patrón correcto simplifica el diseño y reduce errores.
- Cada patrón tiene características de rendimiento predecibles.
- Muchos problemas reales combinan varios patrones.

Patrón Map: operación independiente por elemento

```
1  #pragma omp parallel for
2  for (int i = 0; i < N; i++)
3      c[i] = f(a[i], b[i]);
```

- Cada iteración es independiente: no hay dependencias entre `c[i]` y `c[j]`.
- Conocido como *embarrassingly parallel*: speedup ideal con p *threads* si el cómputo es *compute-bound*.
- Si el cómputo es *memory-bound* (como `c[i]=a[i]+b[i]`), el bus de memoria limita el speedup.

Patrón Reduce: combinar N valores en uno

```
1 double sum = 0.0;
2 #pragma omp parallel for reduction(+:sum)
3 for (int i = 0; i < N; i++)
4     sum += a[i];
```

- OpenMP crea una copia local de `sum` por *thread*.
- Cada *thread* acumula en su copia, sin sincronización.
- Al final de la región paralela, las copias se combinan con `+`.
- Operadores disponibles: `+`, `*`, `-`, `&`, `|`, `^`, `min`, `max`.

Patrón Dividir y Conquistar: tareas recursivas

```
1  void merge_sort(int* a, int n) {
2      if (n <= THRESHOLD) { std::sort(a, a+n); return; }
3
4      #pragma omp task
5      merge_sort(a, n/2);
6      #pragma omp task
7      merge_sort(a + n/2, n - n/2);
8
9      #pragma omp taskwait
10     merge(a, n);
11 }
```

- El árbol de recursión expone paralelismo natural.
- `THRESHOLD` controla la **granularidad**: sin él, millones de tareas mínimas ahogarían el *runtime*.
- El *thread-team* ejecuta tareas a medida que se van encolando.

Patrón de carga irregular: schedule(dynamic)

```
#pragma omp parallel for schedule(dynamic, 1)
for (int i = 0; i < N; i++) {
    resultado[i] = operacion_costosa(i); // costo variable
}
```

- Útil cuando el costo por iteración varía significativamente.
- Con `static`, un *thread* puede terminar pronto y quedar *idle* mientras otros trabajan.
- Con `dynamic`, los *threads* toman iteraciones nuevas a medida que terminan las anteriores.
- El overhead de `dynamic` no es gratis: usar solo cuando el desbalance lo justifica.

Resumen de patrones

Patrón	**Directiva principal**	**Uso**
Map	`parallel for`	Iteraciones independientes
Reduce	`reduction`	Combinar resultados parciales
Dividir y Conquistar	`task + taskwait`	Recursión, árboles
Irregular	`schedule(dynamic)`	Carga variable por iteración
Funcional	`sections`	Tareas heterogéneas

La mayoría de los problemas son combinaciones de estos patrones. Identificar la estructura correcta antes de escribir las directivas reduce los errores.

Midiendo el rendimiento

Factores que afectan la escalabilidad

Patrones de paralelismo

Fuentes de degradación del rendimiento

Límites del paralelismo en memoria compartida

Dos categorías de problemas

Los problemas en código OpenMP se clasifican en dos categorías distintas:

- Correctness : el resultado es incorrecto. Causados por *race conditions*, dependencias de datos ignoradas, o sincronización errónea.
- Performance : el resultado es correcto pero el programa es lento o no escala. Causados por sincronización excesiva, *false sharing*, desbalance de carga, o efectos NUMA.

Un programa incorrecto que parece rápido es el peor escenario: el error puede no manifestarse hasta producción.

Race conditions: la variable compartida invisible

Correctness

El error más común: una variable es `shared` por defecto y varios *threads* la modifican sin sincronización.

```
int counter = 0;
#pragma omp parallel
{
    for (int i = 0; i < 1000; i++)
        counter++;     //  no atómica
}
```

- `counter++` se descompone en 3 instrucciones, no en una.
- Dos *threads* pueden leer el mismo valor, incrementarlo y escribir el mismo resultado: se pierde un incremento.
- Herramienta de diagnóstico: `g++ -fsanitize=thread`, detecta *race conditions* en tiempo de ejecución.

Race condition: coherencia no es suficiente

`counter++` no es atómica: se descompone en tres pasos separados.

Entre el leer y el escribir, otro thread puede intervenir: ambos escriben un resultado válido, pero uno sobreescribe al otro.

Sincronización excesiva

Performance

```
// Lento: un solo hilo activo a la vez
int total = 0;
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    int val = compute(i);
    #pragma omp critical
    total += val;          // todos esperan aquí
}
```

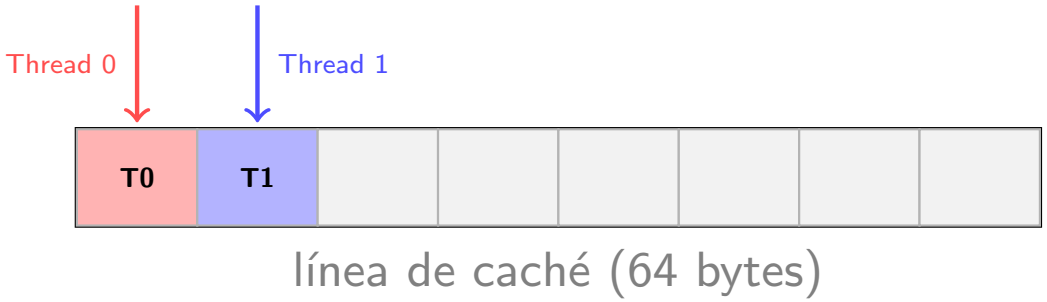
```
// Rápido: reducción paralela
int total = 0;
#pragma omp parallel for reduction(+:total)
for (int i = 0; i < N; i++)
    total += compute(i);  // sin sincronización en el loop
```

`critical` serializa el loop; `reduction` acumula localmente y combina al final.

False sharing: coherencia sin colisión lógica

Performance

Dos *threads* escriben en posiciones distintas de memoria, pero que caen en la misma línea de caché (64 bytes).



Aunque no hay colisión lógica, el protocolo MESI invalida la línea completa cada vez que un *thread* escribe. Los *threads* rebotan la línea entre cachés.

Fix: separar los datos con *padding* a 64 bytes, o usar variables locales al *thread*.

Efectos NUMA: *first-touch policy*

Performance

En sistemas NUMA, la memoria se asigna al nodo del *thread* que la toca primero.

- Inicialización serial: toda la memoria queda en el nodo 0.
- Computación paralela: *threads* en otros nodos acceden a memoria remota, con mayor latencia.

Lento:

- Inicializar arreglo con un loop serial.
- Procesarlo luego en paralelo.

Rápido:

- Inicializar el arreglo en paralelo con el mismo patrón de acceso que usará la computación.

La política *first-touch* significa que inicializar y procesar con el mismo patrón asegura localidad de memoria.

Midiendo el rendimiento

Factores que afectan la escalabilidad

Patrones de paralelismo

Fuentes de degradación del rendimiento

Límites del paralelismo en memoria compartida

La pared del ancho de banda

El paralelismo de datos mejora el throughput de cómputo, pero no el de memoria.

- Todos los *threads* comparten el mismo bus de memoria.
- Para operaciones *memory-bound*, el bus se satura rápidamente.
- Agregar más *threads* a un problema *memory-bound* no reduce el tiempo: el cuello de botella no está en la CPU.

Por ejemplo, `c[i] = a[i] + b[i]` no escala con más *threads*, pero `c[i] = sin(a[i]) + cos(b[i])` sí.

El overhead como trabajo no paralelizable

La parte del trabajo que no escala no es solo el código fuera del `parallel`. Incluye todo lo que introduce serialización:

- Creación y destrucción del *thread-team* (fork-join).
- Sincronización: barreras, *critical*, *atomic*.
- Tráfico de coherencia y *false sharing*: MESI invalida líneas de caché enteras al escribir variables `shared` o datos adyacentes.
- Desbalance de carga: los *threads* rápidos esperan a los lentos en la barrera implícita.

¿Cuándo no usar OpenMP?

OpenMP no es la respuesta universal. Hay situaciones donde paralelizar no ayuda o empeora:

- El problema es *memory-bound*: el bus ya está saturado con un *thread*.
- El loop tiene dependencias entre iteraciones que no se pueden romper fácilmente.
- La sección crítica cubre la mayor parte del trabajo: la fracción serial es alta.
- El problema es demasiado pequeño: el overhead de crear *threads* supera el trabajo.
- El código tiene efectos secundarios no evidentes: I/O, llamadas a funciones no *thread-safe*.

Lo que OpenMP no puede hacer

OpenMP resuelve el paralelismo dentro de una máquina con memoria compartida.

- Cuando el problema no cabe en la memoria de una máquina, o cuando se necesitan múltiples nodos, OpenMP no es suficiente.
- Para paralelismo entre máquinas con memoria distribuida, se usa **MPI** (*Message Passing Interface*).
- MPI requiere comunicación explícita: los procesos no comparten memoria, sino que intercambian mensajes.
- Los dos modelos se pueden combinar: OpenMP dentro de cada nodo, MPI entre nodos.

Cierre del tema: memoria compartida

Durante estas clases trabajamos el modelo de memoria compartida con OpenMP:

- **Modelo**: fork-join, *thread-team*, regiones paralelas.
- **Directivas**: `parallel`, `for`, `sections`, `task`, `reduction`.
- **Sincronización**: `critical`, `atomic`, `barrier`, `taskwait`.
- **Visibilidad de variables**: `private`, `shared`, `firstprivate`, `reduction`.
- **Rendimiento**: escalabilidad, patrones, fuentes de degradación.

El siguiente tema es paralelismo con memoria distribuida: MPI.